

Document number	P0497R0
Date	2016-11-10
Project	Programming Language C++, Library Working Group
Reply-to	Jonathan Wakely <cxx@kayari.org>

Fixes to `shared_ptr` support for arrays

Introduction

LWG Motion 6 in Jacksonville was to apply to the C++ working paper the wording from [P0220R1](#), *Adopt Library Fundamentals V2 TS Components for C++17 (R1)*, but due to conflicts with changes in the C++ WP the `shared_ptr` changes were not applied. [P0414R1](#) remedied that, by clarifying how to apply the changes from the TS to the current draft.

That does not entirely resolve the matter, because the specification in the TS is missing some things (either by accident, or because the specification in the WP changed since a snapshot of it was taken for the TS). There are four issues with `shared_ptr` array support which need to be addressed after P0414R1 is applied to the WP.

Incorrect constraint for `shared_ptr` construction from `unique_ptr`

Originally this constructor was unconstrained and had no requirements. The TS has a requirement added by [N3290][n3290]:

Requires: `Y*` shall be compatible with `T*`.

In parallel, [LWG 2399](#) added a slightly different constraint to the WP:

Remark: This constructor shall not participate in overload resolution unless `unique_ptr<Y, D>::pointer` is convertible to `T*`.

I combined these in P0414R1 as:

Remark: This constructor shall not participate in overload resolution unless `unique_ptr<Y, D>::pointer` is ~~convertible to~~ compatible with `T*`.

Based on implementation experience I believe the correct form is:

Remark: This constructor shall not participate in overload resolution unless `Y*` is compatible with `T*` and `unique_ptr<Y, D>::pointer` is convertible to `element_type*`.

The "compatible with" check prevents undesirable conversions from `unique_ptr<T[]>` to `shared_ptr<T>` and the "convertible to" check ensures that the result of `unique_ptr<Y, D>::get()` can be stored in the `shared_ptr` and returned by `shared_ptr<T>::get()`.

Missing constraint for `weak_ptr` construction from `weak_ptr` rvalues.

[LWG 2315](#) added new `weak_ptr` constructors to the WP which are not present in the TS and so were not adjusted for array support. The fix here is to simply apply the same constraints as for the other `weak_ptr` constructors, requiring that `Y*` is compatible with `T*`.

`shared_ptr<T[]>` and `shared_ptr<T[N]>` comparisons

The definition of `operator<(const shared_ptr<T>& a, const shared_ptr<U>&)` in [util.smartptr.shared.cmp] p2 says:

Returns: `less<V>()(a.get(), b.get())`, where `V` is the composite pointer type (Clause 5) of `T*` and `U*`.

There is no common type for types such as `A(*)[2]` and `B(*)[1]`, so mixed comparisons are not possible.

The definition of `operator<(const shared_ptr<T>& a, nullptr_t)` in [util.smartptr.shared.cmp] p6 says:

Returns: The first function template returns `less<T*>()(a.get(), nullptr)`. The second function template returns `less<T*>()(nullptr, a.get())`.

When `T` is an array type `a.get()` is not convertible to `T*`, so it fails to compile.

Both functions should use `less<element_type*>` instead of simply `T*`.

Interaction with enable_shared_from_this

It doesn't make sense to treat the first element of an array specially, so a `shared_ptr` to an array should not enable `shared_from_this` on construction.

Proposed Wording

Changes are relative to N4606.

Change 20.11.2.2.1 `shared_ptr` constructors [util.smartptr.shared.const]:

```
template<class Y> explicit shared_ptr(Y* p);
```

-4- *Requires:* [...]

-5- *Effects:* Constructs a `shared_ptr` object that owns the pointer `p`. When τ is not an array type, enables `shared_from_this` with `p`. If an exception is thrown, `delete p` is called.

[...]

-8- *Requires:* [...]

-9- *Effects:* Constructs a `shared_ptr` object that owns the object `p` and the deleter `d`. When τ is not an array type, the first and second constructors enable `shared_from_this` with `p`. The second and fourth constructors shall use a copy of `a` to allocate memory for internal use. If an exception is thrown, `d(p)` is called.

[...]

```
template <class Y, class D> shared_ptr(unique_ptr<Y, D>&& r);
```

-26- *Remark:* This constructor shall not participate in overload resolution unless `unique_ptr<Y, D>::pointerY*` is compatible with T^* and `unique_ptr<Y, D>::pointer` is convertible to `element_type*`.

-27 *Effects:* If `r.get() == nullptr`, equivalent to `shared_ptr()`. Otherwise, if `D` is not a reference type, equivalent to `shared_ptr(r.release(), r.get_deleter())`. Otherwise, equivalent to `shared_ptr(r.release(), ref(r.get_deleter()))`. If an exception is thrown, the constructor has no effect. When τ is not an array type, if `r.get() != nullptr`, enables `shared_from_this` with the value that was returned by `r.release()`.

Change 20.11.2.2.7 `shared_ptr` comparison [util.smartptr.shared.cmp]:

```
template<class T, class U> bool operator<(const shared_ptr<T>& a, const shared_ptr<U>& b) noexcept;
```

-2- *Returns:* `less<V>()(a.get(), b.get())`, where `V` is the composite pointer type (Clause 5) of `shared_ptr<T>::element_type*` and `shared_ptr<U>::element_type*`.

[...]

```
template <class T>
bool operator<(const shared_ptr<T>& a, nullptr_t) noexcept;
template <class T>
bool operator<(nullptr_t, const shared_ptr<T>& a) noexcept;
```

-6- *Returns:* The first function template returns `less<shared_ptr<T>::element_type*>()(a.get(), nullptr)`. The second function template returns `less<shared_ptr<T>::element_type*>()(nullptr, a.get())`.

Change 20.11.2.3.1 `weak_ptr` constructors [util.smartptr.weak.const]:

```
weak_ptr(weak_ptr&& r) noexcept;
template<class Y> weak_ptr(weak_ptr<Y>&& r) noexcept;
```

-3- *Remark:* The second constructor shall not participate in overload resolution unless `Y*` is implicitly convertible to `compatible with  $T^*$ .`

Acknowledgments

Thanks to Peter Dimov for reviewing these suggestions.